15-213/18-213，秋季 2012 缓存实验：理解缓存内存分配：2012 年 10 月 2 日星期二，截止日期：2012 年 10 月 11 日星期四晚上 11:59，最后提交时间：2012 年 10 月 14 日星期日晚上 11:59

1 物流

这是一个个人项目。你必须在一个 64 位的 x86-64 机器上运行这个实验。

地点特定：在此处插入任何其他后勤事项，例如如何寻求帮助。

2 概述

本实验将帮助你理解缓存内存对 C 程序性能的影响。
本实验分为两个部分。在第一部分中，你将编写一个小的 C 程序（约 200-300 行），模拟缓存内存的行为。在第二部分中，你将优化一个小型的矩阵转置函数，目标是尽量减少缓存未命中。

3 下载作业

站点特定：在此插入一个段落，解释讲师将如何向学生分发 cachelab-handout.tar 文件。

首先将 cachelab-handout.tar 复制到您计划进行工作的受保护 Linux 目录中。然后输入命令

```
linux> tar xvf cachelab-handout.tar
```

这将创建一个名为 cachelab-handout 的目录，其中包含一些文件。您将修改两个文件：csim.c 和 trans.c。要编译这些文件，请输入：

```
linux> make clean
linux> make
```

您将修改两个文件：csim 警告：不要让 Windows WinZip 程序打开您的 .tar 文件（许多网络浏览器设置为自动执行此操作）。相反，请将文件保存到您的 Linux 目录，并使用 Linux tar 程序提取文件。通常，对于本课程，您绝对不应该使用除 Linux 之外的其他平台来修改您的文件。这样做可能会导致数据丢失（以及重要的工作！）

4 描述

实验分为两部分。在 A 部分，你需要实现一个缓存模拟器。在 B 部分，你需要编写一个针对缓存性能优化的矩阵转置函数。

4.1 参考跟踪文件

手册目录的跟踪子目录包含一组参考跟踪文件，我们将使用它们来评估你在 A 部分编写的缓存模拟器的正确性。这些跟踪文件是由 Linux 程序 valgrind 生成的。例如，输入

```
linux> valgrind --log-fd=1 --tool=lackey -v --trace-mem=yes ls -l
```

在命令行上运行可执行程序“ls -l”，捕获其内存访问的跟踪记录，并按发生顺序打印到标准输出。

Valgrind 内存跟踪的格式如下：

```
I 0400d7d4,8
 M 0421c7f0,4
 L 04f6b868,8
 S 7ff0005c8,8
```

每一行表示一个或两个内存访问。每一行的格式为

[空格]操作地址，大小

操作字段表示内存访问类型：“I”表示指令加载，“L”表示数据加载，“S”表示数据存储，“M”表示数据修改（即数据加载后跟数据存储）。每个“I”前没有空格。每个“M”、“L”和“S”前有空格。地址字段指定一个 64 位十六进制内存地址。大小字段指定操作访问的字节数。

4.2 第一部分：编写缓存模拟器

在第一部分中，你将在 csim.c 中编写一个缓存模拟器，它接受 valgrind 内存跟踪作为输入，模拟缓存内存在此跟踪上的命中/未命中行为，并输出总命中数、未命中数和驱逐数。

我们已经为你提供了一个参考缓存模拟器的二进制可执行文件，称为 csim-ref，它模拟具有任意大小和关联性的缓存的行为，并在 valgrind 跟踪文件上使用 LRU（最不常用）替换策略来选择要驱逐的缓存行。

参考模拟器接受以下命令行参数:

使用方法：./csim-ref [-hv] -s <s> -E <E> -b <b> -t <tracefile>

- -h：可选的帮助标志，打印使用信息
- -v：可选的详细标志，显示跟踪信息
- -s <s>: 组索引位数（S = 2^s 是组数）
- -E <E>：联编性（每组的行数）
- -b <b>: 块位数（B = 2^b 是块大小）
- -t <tracefile>: 要重播的 valgrind 跟踪文件名

命令行参数基于 CS:APP2e 教材第 597 页的 (s, E 和 b) 符号。例如:

```
linux> ./csim-ref -s 4 -E 1 -b 4 -t traces/yi.trace hits:4 misses:5
evictions:3
```

相同的示例在详细模式下的输出:

```
linux> ./csim-ref -v -s 4 -E 1 -b 4 -t traces/yi.trace L 10,1 miss M
20,1 miss hit L 22,1 hit S 18,1 hit
```

```
L 110,1 缺失驱逐 L 210,1 缺失驱逐 M
12,1 缺失命中 命中：4 缺失：5 驱逐：3
```

A 部分的第一项任务是填写 csim.c 文件，使其接受相同的命令行参数并产生与参考模拟器相同的输出。请注意，这个文件几乎完全是空的。
你需要从头开始编写它。

A 部分的编程规则

- 在 csim.c 的头部注释中包含你的姓名和登录 ID。

- 你的 csim.c 文件必须编译无警告才能获得学分。
- 你的模拟器必须能够正确处理任意的 s、E 和 b。这意味着你需要使用 malloc 函数为你的模拟器的数据结构分配存储空间。输入“man malloc”获取该函数的信息。
  输入“man malloc”获取该函数的信息。
- 在本次实验中，我们只关心数据缓存性能，因此你的模拟器应该忽略所有指令缓存访问（以“I”开头的行）。请记住，valgrind 总是将“I”放在第一列（前面没有空格），而将“M”、“L”和“S”放在第二列（前面有空格）。这可能有助于你解析跟踪文件。
- 为了获得 A 部分的成绩，你必须在主函数的末尾调用 printSummary 函数，并传入总的命中次数、未命中次数和驱逐次数。

  打印摘要（hit_count，miss_count，eviction_count）；

- 在这个实验中，你应该假设内存访问是正确对齐的，这样单个内存访问就不会跨越块边界。通过这个假设，你可以忽略 valgrind 跟踪中的请求大小。

4.3 B 部分：优化矩阵转置

在 B 部分，你将在 trans.c 中编写一个转置函数，以尽可能减少缓存未命中。

设 A 表示一个矩阵，A_{ij} 表示该矩阵第 i 行第 j 列的元素。A 的转置矩阵，记作 A^T，是一个矩阵，使得 A^T_{ji} = A_{ij}。
为了帮助你入门，我们提供了一个名为 trans.c 的示例转置函数，该函数计算 N × M 矩阵 A 的转置，并将结果存储在 M × N 矩阵 B 中：

```
char trans_desc[] = "简单的按行扫描转置"；void trans(int M, int N, int
A[N][M], int B[M][N])
```

示例转置函数是正确的，但它效率不高，因为访问模式导致相对较多的缓存未命中。
在 B 部分中，你的任务是编写一个类似的函数，称为 transpose_submit，以最小化不同尺寸矩阵的缓存未命中次数：

```
char transpose_submit_desc[] = "转置提交"; void transpose_submit(int M, int N, int
A[N][M], int B[M][N]);
```

不要更改你的 transpose_submit 函数的描述字符串（“Transpose submission”）。自动评分器会搜索此字符串以确定要评估哪个转置函数以获得学分。

B 部分编程规则

- 在 trans.c 的头注释中包含你的姓名和登录 ID。
- 你的 trans.c 代码必须无警告才能获得学分。
- 每个转置函数中最多允许定义 12 个 int 类型的局部变量。
- 你不允许通过使用 long 类型的变量或使用任何位技巧来将多个值存储到一个变量中，以规避前一条规则。
- 你的转置函数不得使用递归。
- 如果你选择使用辅助函数，那么你的辅助函数和你顶层的转置函数之间栈上的局部变量数量不得超过 12 个。例如，如果你的转置函数声明了 8 个变量，然后你调用一个使用 4 个变量的函数，该函数又调用另一个使用 2 个变量的函数，你将会有 14 个变量在栈上，这将违反规则。
- 你的转置函数不得修改数组 A。然而，你可以对数组 B 的内容做任何你想做的事情。
- 你不被允许在你的代码中定义任何数组或使用 malloc 的任何变体。

5 评估

本节将描述如何评估你的工作。本次实验的满分为 60 分：

- 第一部分：27 分
- 第二部分：26 分
- 样式：7 点

5.1 部分 A 的评估

对于部分 A，我们将使用不同的缓存参数和跟踪来运行您的缓存模拟器。有八个测试用例，每个测试用例 3 分，最后一个用例 6 分：

```
linux> ./csim -s 1 -E 1 -b 1 -t traces/yi2.trace linux> ./csim -s 4
-E 2 -b 4 -t traces/yi.trace linux> ./csim -s 2 -E 1 -b 4 -t
traces/dave.trace linux> ./csim -s 2 -E 1 -b 3 -t
traces/trans.trace linux> ./csim -s 2 -E 2 -b 3 -t
traces/trans.trace
```

[1] 这个限制的原因是，我们的测试代码无法统计对堆栈的引用。我们希望你限制对堆栈的引用，并专注于源数组和目标数组的访问模式。

```
linux> ./csim -s 2 -E 4 -b 3 -t traces/trans.trace linux> ./csim -s
5 -E 1 -b 5 -t traces/trans.trace linux> ./csim -s 5 -E 1 -b 5 -t
traces/long.trace
```

你可以使用参考模拟器 csim-ref 来获得每个测试用例的正确答案。
在调试时，使用-v 选项以获取每个缓存命中和未命中的详细记录。

对于每个测试用例，输出正确的缓存命中、未命中和驱逐次数将为您在该测试用例中获得满分。每个报告的缓存命中、未命中和驱逐次数都占该测试用例分数的 1/3。也就是说，如果一个特定的测试用例值得 3 分，而您的模拟器输出了正确的缓存命中和未命中次数，但报告了错误的驱逐次数，那么您将获得 2 分。

5.2 Part B 的评估

在 B 部分，我们将评估您的 transpose_submit 函数在三种不同尺寸的输出矩阵上的正确性和性能：

- 32 × 32 (M = 32, N = 32)
- 64 × 64 (M = 64, N = 64)
- 61 × 67 (M = 61, N = 67)

5.2.1 性能 (26 分)

对于每个矩阵大小，你的 transpose_submit 函数的性能是通过使用 valgrind 提取你的函数的地址跟踪，然后使用参考模拟器在这个具有参数（s = 5，E = 1，b = 5）的缓存上重播这个跟踪来评估的。
对于每个矩阵大小的你的性能分数随着缺失次数 m 线性增加，直到某个阈值：

- 32 × 32：如果 m < 300，则得 8 分；如果 m > 600，则得 0 分
- 64 × 64：如果 m < 1，则得 8 分，如果 m > 2，000，则得 0 分
- 61 × 67：如果 m < 2，000，则得 10 分，如果 m > 3，000，则得 0 分

您的代码必须正确才能获得特定大小的任何性能分数。您的代码只需要针对这三个情况正确，并且您可以针对这三个情况优化它。特别是，您的函数显式检查输入大小并为每种情况实现单独的优化代码是完全可以的。

5.3 评估风格

代码风格有 7 个评分点。这些将由课程工作人员手动分配。风格指南可以在课程网站上找到。

课程工作人员将在 B 部分检查你的代码，以查找非法数组和过多的局部变量。

6 进行实验

6.1 进行 A 部分

我们已经为您提供了一个自动评分程序，称为 test-csim，该程序在参考跟踪上测试您的缓存模拟器的正确性。在运行测试之前，请确保编译您的模拟器：

```
linux> make
linux> ./test-csim
                                    您的模拟器                  参考模拟器
点 (s,E,b)                  命中 缺失 替换                  命中 缺失 替换
       3 (1,1,1) 9 8 6 9 8                                                    6 跟踪文件/yi2.trace
       3 (4,2,4) 4 5 2 4 5                                                    2 跟踪文件/yi.trace
       3 (2,1,4) 2 3 1 2 3                                                    1 跟踪文件/dave.trace
       3 (2,1,3) 167 71 67 167 71                                            67 跟踪文件/trans.trace
       3 (2,2,3) 201 37 29 201 37                                            29 跟踪/tr.trace
       3 (2,4,3) 212 26 10 212 26                                            10 跟踪/tr.trace
       3 (5,1,5) 231 7 0 231 7                                                0 跟踪/tr.trace
       6 (5,1,5) 265189 21775 21743 265189 21775                          21743 跟踪/长.trace
      27
```

对于每个测试，它显示了您获得的分数、缓存参数、输入跟踪文件以及您的模拟器和参考模拟器结果的比较。

这里有一些关于完成 A 部分的提示和建议：

- 在小跟踪文件上进行初始调试，例如 traces/dave.trace。
- 参考模拟器接受一个可选的 -v 参数，该参数启用详细输出，显示每次内存访问导致的命中、未命中和驱逐。您不需要在您的 csim.c 代码中实现此功能，但我们强烈建议您这样做。它将帮助您通过允许您在参考跟踪文件上直接比较您的模拟器和参考模拟器的行为来进行调试。
- 我们建议您使用 getopt 函数来解析您的命令行参数。您需要以下头文件：

```
#include <getopt.h>
#include <stdlib.h>
#include <unistd.h>
```

参见“man 3 getopt”获取详细信息。

- 每次数据加载（L）或存储（S）操作最多会导致一次缓存未命中。数据修改操作（M）被视为一次加载后存储到同一地址的操作。因此，M 操作可能导致两次缓存命中，或一次未命中加一次命中，并可能发生驱逐。
- 如果你想使用 15-122 的 C0 风格合约，你可以包含我们为方便起见在讲义目录中提供的 contracts.h。

6.2 正在处理 B 部分

我们提供了一个自动评分程序，名为 test-trans.c，用于测试你注册的每个转置函数的正确性和性能。
您可以在您的 trans.c 文件中注册最多 100 个转置函数版本。每个转置函数版本具有以下形式：

```
/* 头部注释 */ char trans_simple_desc[] = "一个简单的转置"; void
trans_simple(int M, int N, int A[N][M], int B[M][N]) {

/* 您的转置代码在这里 */ }
```

通过以下形式的调用向自动评分器注册特定的转置函数：

```
注册转换函数(trans_simple, trans_simple_desc);
```

在 trans.c 中的 registerFunctions 例程中。在运行时，自动评分器将评估每个注册的转置函数并打印结果。当然，注册的函数中必须有一个你提交用于计分的 transpose_submit 函数：

```
注册转换函数(transpose_submit, transpose_submit_desc);
```

请参阅默认的 trans.c 函数，了解如何工作。

自动评分器将矩阵大小作为输入。它使用 valgrind 生成每个注册的转置函数的跟踪记录。然后，它通过在具有参数（s = 5，E = 1，b = 5）的缓存上运行参考模拟器来评估每个跟踪记录。
例如，要在 32 × 32 矩阵上测试您的注册转置函数，请重新构建 test-trans，然后使用适当的 M 和 N 值运行它：

```
linux> make linux> ./test-trans -M 32 -N 32 第一步：评估注册的转置函数的正确性：
func 0（提交的转置）：正确性：1
```

/test-trans -M 32 -N 32 第一步：评估注册的转置函数的正确性：func 0（转置提交）：正确性：1 func 1（简单的按行扫描转置）：正确性：1 func 2（按列扫描转置）：正确性：1 func 3（使用锯齿形访问模式）：正确性：1

第 2 步：为注册的转置函数生成内存跟踪。

第 3 步：评估注册的转置函数的性能（s=5, E=1, b=5）函数 0（转置提交）：命中：1766，未命中：287，驱逐：255 函数 1（简单行扫描转置）：命中：870，未命中：1183，驱逐：1151 函数 2（列扫描转置）：命中：870，未命中：1183，驱逐：1151 函数 3（使用锯齿形访问模式）：命中：1076，未命中：977，驱逐：945

官方提交（函数 0）的摘要：正确性=1 未命中=287

在这个示例中，我们在 trans.c 中注册了四个不同的转置函数。test-trans 程序测试每个注册的函数，显示每个函数的结果，并提取官方提交的结果。

这里有一些关于完成 B 部分的提示和建议。

- 测试程序 test-trans 将函数 i 的跟踪信息保存到文件 trace.fi 中。这些跟踪文件是非常有价值的调试工具，可以帮助你了解每个转置函数的命中和未命中情况。要调试某个特定的函数，只需将其跟踪信息通过参考模拟器并使用详细选项运行：

  ```
  linux> ./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0 S 68312c,1 未命
  中 L 683140,8 未命中

  L 683124,4 命中
  L 683120,4 命中 L 603124,4 未命中
  替换 S 6431a0,4 未命中 ...
  ```

- 由于你的转置函数在直接映射缓存上被评估，冲突未命中是一个潜在问题。考虑你的代码中冲突未命中的可能性，特别是在对角线上。尝试思考一些可以减少这些冲突未命中数量的访问模式。

- 阻塞是一种减少缓存未命中的有用技术。参见

  http://csapp.cs.cmu.edu/public/waside/waside-blocking.pdf

  获取更多信息。

因为 valgrind 引入了许多与你的代码无关的栈访问，我们已经从跟踪中过滤掉了所有栈访问。这就是为什么我们禁止了局部数组，并对局部变量的数量进行了限制。

6.3 将所有内容整合在一起

我们已经为您提供了一个驱动程序，名为 ./driver.py，它对您的模拟器和转置代码进行完整评估。这是您的讲师用来评估您提交作业的程序。驱动程序使用 test-csim 来评估您的模拟器，并使用 test-trans 来评估您提交的转置函数在三种矩阵大小上的表现。然后它打印出您的结果摘要和您获得的分数。

要运行驱动程序，请输入：

```
linux> ./driver.py
```

7 提交你的作业

每次你在 cachelab-handout 目录下输入 make，Makefile 都会创建一个名为 userid-handin.tar 的归档文件，其中包含你当前的 csim.c 和 trans.c 文件。

学校特定：在此处插入文本，告诉每位学生如何在你的学校提交 useridhandin.tar 文件。

重要提示：不要在 Windows 或 Mac 机器上创建提交用的 tar 归档文件，并且不要以任何其他归档格式（如 .zip、.gzip 或 .tgz 文件）提交文件。